

# Índice

## **1. Presentación**

## **2. Índice**

## **3. Introducción**

- Jerarquía de clases
- Diseño

## **4. Ejercicio 1 resuelto**

- Código fuente de las clases implementadas
- Código de las pruebas unitarias (JUnit 5)
- Resultados de ejecución de pruebas

## **5. Ejercicio 2 resuelto**

- Cálculo de complejidad ciclomática
- Caminos básicos del código

## **6. Ejercicio 3 resuelto**

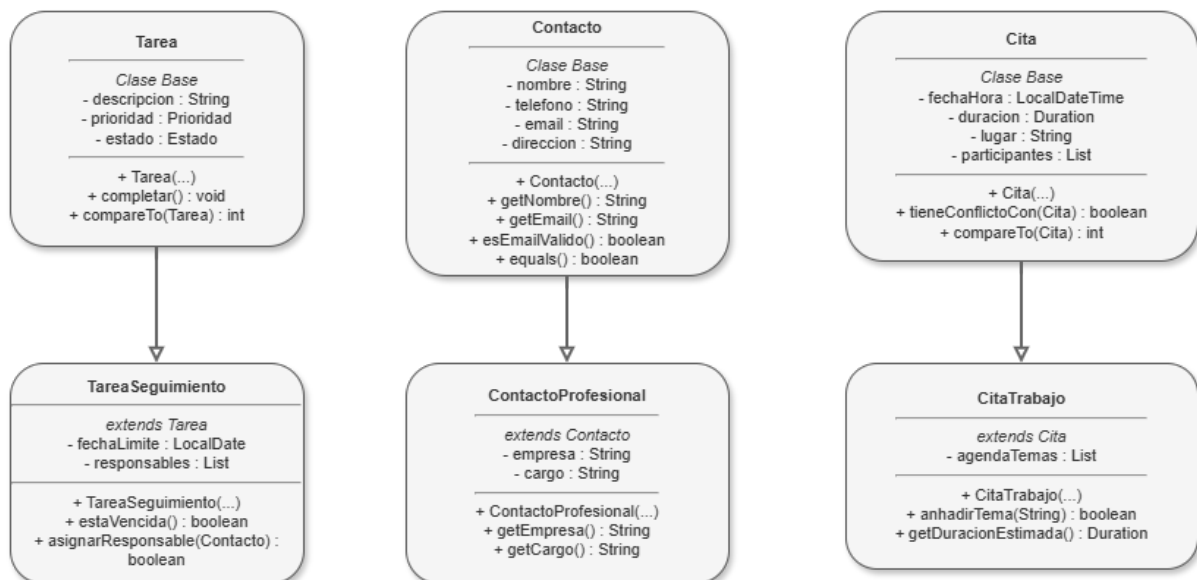
- Diseño de alto nivel mediante ingeniería inversa
- Resultados y justificación de pruebas de caja blanca
- Resultados y justificación de pruebas de caja negra

# Introducción

Para esta práctica, primeramente necesitamos implementar un modelo de dominio para una agenda digital que gestiona contactos, citas y tareas pendientes. Siendo estas tres las ideas principales del modelo, para la implementación plantearemos tres clases (Contacto, Cita y Tarea) las cuales se relacionarán y construirán por herencia un segundo conjunto de entidades especializadas (Contactos profesionales, Citas de trabajo y Tareas con seguimiento).

## Jerarquía de clases

- Contacto -> ContactoProfesional (añade empresa y cargo).
- Cita -> CitaTrabajo (añade agenda de temas y duración estimada).
- Tarea -> TareaSeguimiento (añade fecha límite y responsables asignados).



## Diseño

Para el diseño de las clases aplicamos lo básico de Clean Code y SOLID sin complicarnos demasiado:

- Validamos los datos en los setters y los marcamos como final para que las subclases no pudieran saltarse las reglas.
- Usamos enum para la prioridad y el estado de las tareas, así evitamos valores inconsistentes. Las colecciones que devolvemos (participantes, responsables, temas) las exponemos con `Collections.unmodifiableList()` para que no se modifiquen desde fuera sin control.

- En los contactos, sobrescribimos equals y hashCode basándonos solo en el email, porque lo consideramos su identificador único dentro de la agenda.
- Para la detección de conflictos entre citas usamos intervalos semi-abiertos [inicio, fin), de forma que dos citas seguidas (una termina justo cuando empieza la otra) no se marquen como conflicto.
- El validador de email lo implementamos como una clase de utilidad con constructor privado que lanza AssertionError, siguiendo la recomendación de Bloch.
- Por último, en los constructores que reciben listas permitimos que se pase null y lo sustituimos por List.of(), así evitamos tener que comprobar nulos a cada rato.

# Ejercicio 1 resuelto

## Código fuente de las clases implementadas

### - Prioridad.java

```
package es.uah.agenda.modelo;

public enum Prioridad {
    BAJA(1), MEDIA(2), ALTA(3), CRITICA(4);

    private final int nivel;

    Prioridad(int nivel) {
        this.nivel = nivel;
    }

    public int getNivel() {
        return nivel;
    }
}
```

### - Estado.java

```
package es.uah.agenda.modelo;

public enum Estado {

    PENDIENTE, EN_PROGRESO, COMPLETADA, CANCELADA

}
```

### - ValidadorEmail.java

```
package es.uah.agenda.util;

import java.util.regex.Pattern;

public final class ValidadorEmail {

    // RFC 5322 simplificado: cubre la mayoría de casos reales sin complejidad excesiva

    private static final Pattern PATRON_EMAIL = Pattern.compile(

        "^[a-zA-Z0-9_+&*-]+(?:\\.[a-zA-Z0-9_+&*-]+)*@" +
```

```

        "([a-zA-Z0-9-]+\\.)+[a-zA-Z]{2,7}$"

    );

    private ValidadorEmail() {

        throw new AssertionError("Clase de utilidad, no instanciable");

    }

    public static boolean esValido(String email) {

        return email != null && PATRON_EMAIL.matcher(email).matches();

    }

}

```

## - Contacto.java

```

package es.uah.agenda.modelo;

import es.uah.agenda.util.ValidadorEmail;
import java.util.Objects;

public class Contacto {

    private String nombre;
    private String telefono;
    private String email;
    private String direccion;

    public Contacto(String nombre, String telefono, String email, String direccion) {
        setNombre(nombre);
        setTelefono(telefono);
        setEmail(email);
        this.direccion = direccion;
    }

    public final void setNombre(String nombre) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede ser nulo o vacío");
        }
        this.nombre = nombre.trim();
    }

    public final void setTelefono(String telefono) {
        if (telefono == null || !telefono.matches("^\\+?[0-9]{9,15}$")) {
            throw new IllegalArgumentException("Teléfono inválido: " + telefono);
        }
        this.telefono = telefono;
    }

    public final void setEmail(String email) {
        if (!ValidadorEmail.esValido(email)) {
            throw new IllegalArgumentException("Email inválido: " + email);
        }
    }
}

```

```

    }
    this.email = email;
}

public void setDireccion(String direccion) {
    this.direccion = direccion;
}

public String getNombre() { return nombre; }
public String getTelefono() { return telefono; }
public String getEmail() { return email; }
public String getDireccion() { return direccion; }

public boolean esEmailValido() {
    return ValidadorEmail.esValido(this.email);
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Contacto c)) return false;
    return Objects.equals(email, c.email);
}

@Override
public int hashCode() {
    return Objects.hash(email);
}

@Override
public String toString() {
    return "Contacto{nombre='%s', email='%s'}.formatted(nombre, email);
}
}

```

## - ContactoProfesional.java

```

package es.uah.agenda.modelo;

public class ContactoProfesional extends Contacto {

    private String empresa;

    private String cargo;

    public ContactoProfesional(String nombre, String telefono, String email,
                                String direccion, String empresa, String cargo) {

        super(nombre, telefono, email, direccion);

        setEmpresa(empresa);
    }
}

```

```

        setCargo(cargo);
    }

    public final void setEmpresa(String empresa) {

        if (empresa == null || empresa.isBlank()) {

            throw new IllegalArgumentException("Empresa no puede ser nula o vacía");

        }

        this.empresa = empresa.trim();
    }

    public final void setCargo(String cargo) {

        if (cargo == null || cargo.isBlank()) {

            throw new IllegalArgumentException("Cargo no puede ser nulo o vacío");

        }

        this.cargo = cargo.trim();
    }

    public String getEmpresa() { return empresa; }

    public String getCargo() { return cargo; }

    @Override

    public String toString() {

        return "ContactoProfesional{nombre='%s', empresa='%s', cargo='%s'}"

            .formatted(getNombre(), empresa, cargo);

    }
}

```

## - Cita.java

```
package es.uah.agenda.modelo;
```

```
import java.time.Duration;
```

```
import java.time.LocalDateTime;
```

```
import java.util.ArrayList;
```

```
import java.util.Collections;
```

```
import java.util.List;
```

```
import java.util.Objects;
```

```
public class Cita implements Comparable<Cita> {
```

```
    private LocalDateTime fechaHora;
```

```
    private Duration duracion;
```

```
    private String lugar;
```

```
    private final List<Contacto> participantes;
```

```
    public Cita(LocalDateTime fechaHora, Duration duracion, String lugar, List<Contacto> participantes) {
```

```
        setFechaHora(fechaHora);
```

```
        setDuracion(duracion);
```

```
        setLugar(lugar);
```

```
        this.participantes = new ArrayList<>(Objects.requireNonNullElse(participantes, List.of()));
```

```
    }
```

```
    public final void setFechaHora(LocalDateTime fechaHora) {
```

```
        if (fechaHora == null) {
```

```
            throw new IllegalArgumentException("La fecha/hora no puede ser nula");
```

```
        }
```

```
        this.fechaHora = fechaHora;
```

```
    }
```

```

public final void setDuracion(Duration duracion) {

    if (duracion == null || duracion.isNegative() || duracion.isZero()) {

        throw new IllegalArgumentException("Duración debe ser positiva");

    }

    this.duracion = duracion;

}


public final void setLugar(String lugar) {

    if (lugar == null || lugar.isBlank()) {

        throw new IllegalArgumentException("El lugar no puede ser nulo o vacío");

    }

    this.lugar = lugar.trim();

}


public LocalDateTime getFechaHora()    { return fechaHora; }

public Duration getDuracion()          { return duracion; }

public String getLugar()                { return lugar; }

public List<Contacto> getParticipantes() { return Collections.unmodifiableList(participantes); }


public LocalDateTime getFechaFin() {

    return fechaHora.plus(duracion);

}


public boolean añadirParticipante(Contacto c) {

    return c != null && !participantes.contains(c) && participantes.add(c);

}


public boolean eliminarParticipante(Contacto c) {

```

```
        return participantes.remove(c);  
    }  
}
```

```
public boolean tieneConflictoCon(Cita otra) {  
    if (otra == null || otra == this) return false;  
    return this.fechaHora.isBefore(otra.getFechaFin())  
        && otra.getFechaHora().isBefore(this.getFechaFin());  
}
```

```
@Override  
public int compareTo(Cita otra) {  
    return this.fechaHora.compareTo(otra.fechaHora);  
}
```

```
@Override  
public String toString() {  
    return "Cita{fecha=%s, lugar='%s', duracion=%dmin}"  
        .formatted(fechaHora, lugar, duracion.toMinutes());  
}  
}
```

## - CitaTrabajo.java

```
package es.uah.agenda.modelo;
```

```
import java.time.Duration;
```

```
import java.time.LocalDateTime;
```

```
import java.util.ArrayList;
```

```
import java.util.Collections;
```

```
import java.util.List;
```

```

public class CitaTrabajo extends Cita {

    private final List<String> agendaTemas;

    public CitaTrabajo(LocalDateTime fechaHora, Duration duracion, String lugar,
        List<Contacto> participantes, List<String> agendaTemas) {

        super(fechaHora, duracion, lugar, participantes);

        this.agendaTemas = new ArrayList<>(agendaTemas != null ? agendaTemas : List.of());
    }

    public boolean añadirTema(String tema) {

        if (tema == null || tema.isBlank()) return false;

        return agendaTemas.add(tema.trim());
    }

    public boolean eliminarTema(String tema) {

        return agendaTemas.remove(tema);
    }

    public List<String> getAgendaTemas() {

        return Collections.unmodifiableList(agendaTemas);
    }

    /** Duración estimada = duración programada de la cita. */

    public Duration getDuracionEstimada() {

        return getDuracion();
    }

    @Override

    public String toString() {

```

```

        return "CitaTrabajo{fecha=%s, lugar='%s', temas=%d}"
            .formatted(getFechaHora(), getLugar(), agendaTemas.size());
    }
}

```

## - Tarea.java

```

package es.uah.agenda.modelo;

import java.util.Comparator;
import java.util.Objects;

public class Tarea implements Comparable<Tarea> {

    // Orden: CRITICA (4) primero → BAJA (1) último

    public static final Comparator<Tarea> POR_PRIORIDAD_DESC =
        Comparator.comparingInt((Tarea t) -> t.prioridad.getNivel()).reversed();

    private String descripcion;

    private Prioridad prioridad;

    private Estado estado;

    public Tarea(String descripcion, Prioridad prioridad, Estado estado) {

        setDescripcion(descripcion);

        setPrioridad(prioridad);

        setEstado(estado);
    }

    public final void setDescripcion(String descripcion) {

        if (descripcion == null || descripcion.isBlank()) {

            throw new IllegalArgumentException("Descripción no puede ser nula o vacía");

        }
    }
}

```

```

        this.descripcion = descripcion.trim();
    }

    public final void setPrioridad(Prioridad prioridad) {

        this.prioridad = Objects.requireNonNull(prioridad, "Prioridad no puede ser nula");
    }

    public final void setEstado(Estado estado) {

        this.estado = Objects.requireNonNull(estado, "Estado no puede ser nulo");
    }

    public String getDescripcion() { return descripcion; }

    public Prioridad getPrioridad() { return prioridad; }

    public Estado getEstado() { return estado; }

    public void completar() {

        this.estado = Estado.COMPLETADA;
    }

    /** Orden natural: por prioridad descendente. */

    @Override

    public int compareTo(Tarea otra) {

        return POR_PRIORIDAD_DESC.compare(this, otra);
    }

    @Override

    public String toString() {

        return "Tarea{desc='%s', prio=%s, estado=%s}"

            .formatted(descripcion, prioridad, estado);
    }
}

```

## - TareaSeguimiento.java

```
package es.uah.agenda.modelo;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Objects;

public class TareaSeguimiento extends Tarea {

    private LocalDate fechaLimite;
    private final List<Contacto> responsables;

    public TareaSeguimiento(String descripcion, Prioridad prioridad, Estado estado,
        LocalDate fechaLimite, List<Contacto> responsables) {
        super(descripcion, prioridad, estado);
        setFechaLimite(fechaLimite);
        this.responsables = new ArrayList<>(Objects.requireNonNullElse(responsables, List.of()));
    }

    public final void setFechaLimite(LocalDate fechaLimite) {
        this.fechaLimite = Objects.requireNonNull(fechaLimite, "Fecha límite no puede ser nula");
    }

    public LocalDate getFechaLimite() { return fechaLimite; }

    public List<Contacto> getResponsables() {
        return Collections.unmodifiableList(responsables);
    }

    public boolean asignarResponsable(Contacto c) {
        return c != null && !responsables.contains(c) && responsables.add(c);
    }

    public boolean eliminarResponsable(Contacto c) {
        return responsables.remove(c);
    }

    /** Una tarea está vencida si hoy > fechaLimite y no está completada/cancelada. */
    public boolean estaVencida() {
        if (getEstado() == Estado.COMPLETADA || getEstado() == Estado.CANCELADA) {
            return false;
        }
        return LocalDate.now().isAfter(fechaLimite);
    }

    @Override
    public String toString() {
        return "TareaSeguimiento{desc='%s', prio=%s, limite=%s, responsables=%d}"
            .formatted(getDescripcion(), getPrioridad(), fechaLimite, responsables.size());
    }
}
```

## - Main.java

```
import es.uah.agenda.modelo.*;

import java.time.*;

import java.util.List;

public class Main {

    public static void main(String[] args) {

        Contacto ana = new Contacto("Ana", "+34600111222", "ana@uah.es", "C/ Mayor 1");

        System.out.println(ana);

        Cita c1 = new Cita(LocalDateTime.of(2026, 5, 1, 10, 0),

            Duration.ofHours(2), "Sala A", List.of(ana));

        Cita c2 = new Cita(LocalDateTime.of(2026, 5, 1, 11, 0),

            Duration.ofHours(1), "Sala B", List.of());

        System.out.println("¿ Conflicto? " + c1.tieneConflictoCon(c2)); // true

    }

}
```

## Código de las pruebas unitarias (JUnit 5)

Hemos realizado 35 pruebas con JUnit 5 (proporcionado por el IDE) entre las 6 clases. El objetivo de cada prueba es comprobar que se satisfacen los requisitos necesarios para cada una de las clases definidas anteriormente.

## - Citatest

```
package es.uah.agenda.modelo;

import org.junit.jupiter.api.BeforeEach;

import org.junit.jupiter.api.Test;
```

```
import java.time.Duration;

import java.time.LocalDateTime;

import java.util.List;


import static org.junit.jupiter.api.Assertions.*;


class CitaTest {


    private Contacto c1;

    private LocalDateTime base;


    @BeforeEach

    void setUp() {

        c1 = new Contacto("Ana", "+34600111222", "ana@uah.es", "dir");

        base = LocalDateTime.of(2026, 5, 1, 10, 0);

    }


    @Test

    void creacionValida() {

        Cita cita = new Cita(base, Duration.ofHours(1), "Sala A", List.of(c1));

        assertEquals(base, cita.getFechaHora());

        assertEquals(Duration.ofHours(1), cita.getDuracion());

        assertEquals(1, cita.getParticipantes().size());

    }


    @Test

    void duracionNoPositivaDebeFallar() {

        assertThrows(IllegalArgumentException.class,

            () -> new Cita(base, Duration.ZERO, "Sala", List.of()));

        assertThrows(IllegalArgumentException.class,

            () -> new Cita(base, Duration.ofMinutes(-30), "Sala", List.of()));

    }

}
```

```
}
```

```
@Test
```

```
void conflictoDeSolapamientoParcial() {
```

```
    Cita a = new Cita(base, Duration.ofHours(2), "S1", List.of());
```

```
    Cita b = new Cita(base.plusHours(1), Duration.ofHours(2), "S2", List.of());
```

```
    assertTrue(a.tieneConflictoCon(b));
```

```
    assertTrue(b.tieneConflictoCon(a));
```

```
}
```

```
@Test
```

```
void sinConflictoAdyacentes() {
```

```
    Cita a = new Cita(base, Duration.ofHours(1), "S1", List.of());
```

```
    Cita b = new Cita(base.plusHours(1), Duration.ofHours(1), "S2", List.of());
```

```
    assertFalse(a.tieneConflictoCon(b),
```

```
        "Fin exacto = inicio siguiente NO debe ser conflicto (intervalos semi-abiertos)");
```

```
}
```

```
@Test
```

```
void sinConflictoSeparadas() {
```

```
    Cita a = new Cita(base, Duration.ofHours(1), "S1", List.of());
```

```
    Cita b = new Cita(base.plusHours(3), Duration.ofHours(1), "S2", List.of());
```

```
    assertFalse(a.tieneConflictoCon(b));
```

```
}
```

```
@Test
```

```
void noConflictoConsigoMismaNiConNull() {
```

```
    Cita a = new Cita(base, Duration.ofHours(1), "S1", List.of());
```

```
    assertFalse(a.tieneConflictoCon(a));
```

```
    assertFalse(a.tieneConflictoCon(null));
```

```
}
```

@Test

```
void añadirParticipanteEvitaDuplicados() {  
  
    Cita cita = new Cita(base, Duration.ofHours(1), "S1", List.of());  
  
    assertTrue(cita.añadirParticipante(c1));  
  
    assertFalse(cita.añadirParticipante(c1));  
  
    assertEquals(1, cita.getParticipantes().size());  
  
}
```

@Test

```
void ordenNaturalPorFecha() {  
  
    Cita a = new Cita(base.plusHours(5), Duration.ofHours(1), "S1", List.of());  
  
    Cita b = new Cita(base, Duration.ofHours(1), "S2", List.of());  
  
    assertTrue(a.compareTo(b) > 0);  
  
}  
}
```

## REQUISITOS CLASE CITA

- La cita debe crearse con fecha, duración, lugar y participantes válidos
- La duración no puede ser cero ni negativa
- Dos citas que se solapan parcialmente deben detectar conflicto
- Dos citas consecutivas (una termina justo cuando empieza la otra) no deben detectar conflicto
- Dos citas separadas en el tiempo no deben detectar conflicto
- Una cita no debe detectar conflicto consigo misma ni con null
- No se pueden añadir participantes duplicados a una cita
- Las citas deben ordenarse por fecha de forma natural (compareTo)

### - CitaTrabajoTest

```
package es.uah.agenda.modelo;
```

```
import org.junit.jupiter.api.Test;
```

```
import java.time.Duration;
```

```
import java.time.LocalDateTime;
```

```
import java.util.List;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
class CitaTrabajoTest {
```

```
    @Test
```

```
    void gestionDeTemas() {
```

```
        CitaTrabajo ct = new CitaTrabajo(
```

```
            LocalDateTime.of(2026, 5, 1, 9, 0),
```

```
            Duration.ofMinutes(90),
```

```
            "Sala Reuniones",
```

```
            List.of(),
```

```
            List.of("Q2 Review")
```

```
        );
```

```
        assertTrue(ct.añadirTema("Roadmap H2"));
```

```
        assertFalse(ct.añadirTema(" "));
```

```
        assertFalse(ct.añadirTema(null));
```

```
        assertEquals(2, ct.getAgendaTemas().size());
```

```
        assertTrue(ct.eliminarTema("Q2 Review"));
```

```
    }
```

```
    @Test
```

```
    void listaDeTemasEsInmutable() {
```

```
        CitaTrabajo ct = new CitaTrabajo(
```

```
            LocalDateTime.now(), Duration.ofHours(1), "S", List.of(), List.of("t1")
```

```

    );

    assertThrows(UnsupportedOperationException.class,

        () -> ct.getAgendaTemas().add("hack"));
    }

    @Test
    void duracionEstimadaCoincideConDuracion() {

        CitaTrabajo ct = new CitaTrabajo(

            LocalDateTime.now(), Duration.ofMinutes(45), "S", List.of(), List.of()

        );

        assertEquals(Duration.ofMinutes(45), ct.getDuracionEstimada());
    }
}

```

## REQUISITOS CLASE CITA TRABAJO

- Se deben poder añadir y eliminar temas de la agenda de trabajo
- Los temas vacíos o nulos no deben añadirse
- La lista de temas expuesta debe ser inmutable
- La duración estimada debe coincidir con la duración programada de la cita

### - ContactoProfesionalTest

```

package es.uah.agenda.modelo;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class ContactoProfesionalTest {

    @Test

```

```

void creacionValida() {

    ContactoProfesional cp = new ContactoProfesional(

        "Luis", "+34699888777", "luis@acme.com", "Av. Principal 10",

        "ACME", "CTO"

    );

    assertEquals("ACME", cp.getEmpresa());

    assertEquals("CTO", cp.getCargo());

    assertTrue(cp instanceof Contacto);

}

@Test

void empresaVacíaDebeFallar() {

    assertThrows(IllegalArgumentException.class, () -> new ContactoProfesional(

        "Luis", "+34699888777", "luis@acme.com", "dir", " ", "CTO"));

}

@Test

void cargoNuloDebeFallar() {

    assertThrows(IllegalArgumentException.class, () -> new ContactoProfesional(

        "Luis", "+34699888777", "luis@acme.com", "dir", "ACME", null));

}

}

```

## REQUISITOS CONTACTO PROFESIONAL

- Un contacto profesional debe heredar todos los atributos de Contacto
- La empresa no puede ser nula ni estar vacía
- El cargo no puede ser nulo ni estar vacío

## - ContactoTest

```
package es.uah.agenda.modelo;
```

```

import org.junit.jupiter.api.Test;

import org.junit.jupiter.params.ParameterizedTest;

import org.junit.jupiter.params.provider.ValueSource;


import static org.junit.jupiter.api.Assertions.*;


class ContactoTest {

    @Test

    void creacionValidaDebeAsignarCampos() {

        Contacto c = new Contacto("Ana", "+34600111222", "ana@uah.es", "C/ Mayor 1");

        assertEquals("Ana", c.getNombre());

        assertEquals("ana@uah.es", c.getEmail());

        assertTrue(c.esEmailValido());

    }


    @ParameterizedTest

    @ValueSource(strings = {"sin-arroba", "a@b", "@uah.es", "espacio @uah.es"})

    void emailsInvalidosDebenLanzarExcepcion(String email) {

        assertThrows(IllegalArgumentException.class,

            () -> new Contacto("Ana", "+34600111222", email, "dir"));

    }


    @ParameterizedTest

    @ValueSource(strings = {"123", "abcdefghij", "+34 600 111 222"})

    void telefonosInvalidosDebenLanzarExcepcion(String tel) {

        assertThrows(IllegalArgumentException.class,

            () -> new Contacto("Ana", tel, "ana@uah.es", "dir"));

    }
}

```

@Test

```
void nombreNuloOVacioDebeLanzarExcepcion() {  
  
    assertThrows(IllegalArgumentException.class,  
  
        () -> new Contacto(null, "+34600111222", "ana@uah.es", "dir"));  
  
    assertThrows(IllegalArgumentException.class,  
  
        () -> new Contacto(" ", "+34600111222", "ana@uah.es", "dir"));  
  
}
```

@Test

```
void equalsBasadoEnEmail() {  
  
    Contacto c1 = new Contacto("Ana", "+34600111222", "ana@uah.es", "d1");  
  
    Contacto c2 = new Contacto("Ana Bis", "+34600333444", "ana@uah.es", "d2");  
  
    assertEquals(c1, c2);  
  
    assertEquals(c1.hashCode(), c2.hashCode());  
  
}  
}
```

## REQUISITOS CLASE CONTACTO

- El contacto debe poder crearse con datos válidos
- El email debe tener un formato correcto (RFC 5322 simplificado)
- El teléfono debe tener entre 9 y 15 dígitos y puede empezar con +
- El nombre no puede ser nulo ni estar vacío
- Dos contactos con el mismo email deben considerarse iguales (mismo hashCode)

### - TareaSeguimientoTest

```
package es.uah.agenda.modelo;
```

```
import org.junit.jupiter.api.Test;
```

```
import java.time.LocalDate;
```

```
import java.util.List;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
class TareaSeguimientoTest {
```

```
    private Contacto resp() {  
        return new Contacto("Ana", "+34600111222", "ana@uah.es", "d");  
    }  
}
```

```
@Test
```

```
void creacionValida() {  
    TareaSeguimiento ts = new TareaSeguimiento(  
        "Entregar informe", Prioridad.ALTA, Estado.PENDIENTE,  
        LocalDate.now().plusDays(3), List.of(resp())  
    );  
    assertEquals(1, ts.getResponsables().size());  
    assertFalse(ts.estaVencida());  
}
```

```
@Test
```

```
void tareaPendienteVencidaSiFechaEnPasado() {  
    TareaSeguimiento ts = new TareaSeguimiento(  
        "X", Prioridad.MEDIA, Estado.PENDIENTE,  
        LocalDate.now().minusDays(1), List.of()  
    );  
    assertTrue(ts.estaVencida());  
}
```

```
@Test
```

```
void tareaCompletadaNoEstaVencidaAunqueFechaPasada() {
```

```

TareaSeguimiento ts = new TareaSeguimiento(
    "X", Prioridad.MEDIA, Estado.COMPLETADA,
    LocalDate.now().minusDays(5), List.of()
);

assertFalse(ts.estaVencida());
}

```

@Test

```

void tareaCanceladaNoEstaVencida() {
    TareaSeguimiento ts = new TareaSeguimiento(
        "X", Prioridad.MEDIA, Estado.CANCELADA,
        LocalDate.now().minusDays(5), List.of()
    );

    assertFalse(ts.estaVencida());
}

```

@Test

```

void asignacionResponsablesEvitaDuplicados() {
    Contacto c = resp();

    TareaSeguimiento ts = new TareaSeguimiento(
        "X", Prioridad.BAJA, Estado.PENDIENTE, LocalDate.now().plusDays(1), List.of()
    );

    assertTrue(ts.asignarResponsable(c));
    assertFalse(ts.asignarResponsable(c));
    assertEquals(1, ts.getResponsables().size());
}

```

@Test

```

void fechaLimiteNulaDebeFallar() {
    assertThrows(NullPointerException.class, () -> new TareaSeguimiento(
        "X", Prioridad.BAJA, Estado.PENDIENTE, null, List.of()
    ));
}

```

```
    ));  
}  
}
```

## REQUISITOS TAREA SEGUIMIENTO

- Una tarea con seguimiento debe poder tener responsables asignados
- Una tarea pendiente con fecha límite pasada debe estar vencida
- Una tarea completada NO debe estar vencida aunque su fecha límite haya pasado
- Una tarea cancelada NO debe estar vencida aunque su fecha límite haya pasado
- No se pueden asignar responsables duplicados
- La fecha límite no puede ser nula

## - TareaTest

```
package es.uah.agenda.modelo;
```

```
import org.junit.jupiter.api.Test;
```

```
import java.util.ArrayList;
```

```
import java.util.Collections;
```

```
import java.util.List;
```

```
import static org.junit.jupiter.api.Assertions.*;
```

```
class TareaTest {
```

```
    @Test
```

```
    void creacionValida() {
```

```
        Tarea t = new Tarea("Revisar PR", Prioridad.ALTA, Estado.PENDIENTE);
```

```

        assertEquals(Prioridad.ALTA, t.getPrioridad());

        assertEquals(Estado.PENDIENTE, t.getEstado());

    }

    @Test

    void completarCambiaEstado() {

        Tarea t = new Tarea("X", Prioridad.BAJA, Estado.EN_PROGRESO);

        t.completar();
    }

```

## REQUISITOS CLASE TAREA

- La tarea debe crearse con descripción, prioridad y estado válidos
- La descripción no puede ser nula ni vacía
- La prioridad no puede ser nula
- El estado no puede ser nulo
- Al completar una tarea, su estado debe cambiar a COMPLETADA

## Resultados de ejecución de pruebas

Ejecutamos los tests con Maven y JUnit 5. Los 35 tests se ejecutan bien. Sin fallos, errores ni tests omitidos.

### TESTS

Running es.uah.agenda.modelo.CitaTest

Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 s -- in es.uah.agenda.modelo.CitaTest

Running es.uah.agenda.modelo.CitaTrabajoTest

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.022 s -- in es.uah.agenda.modelo.CitaTrabajoTest

Running es.uah.agenda.modelo.ContactoProfesionalTest

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in es.uah.agenda.modelo.ContactoProfesionalTest

Running es.uah.agenda.modelo.ContactoTest

Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.073 s -- in es.uah.agenda.modelo.ContactoTest

Running es.uah.agenda.modelo.TareaSeguimientoTest

Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in es.uah.agenda.modelo.TareaSeguimientoTest

Running es.uah.agenda.modelo.TareaTest

Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in es.uah.agenda.modelo.TareaTest

Results:

Tests run: 35, Failures: 0, Errors: 0, Skipped: 0

---

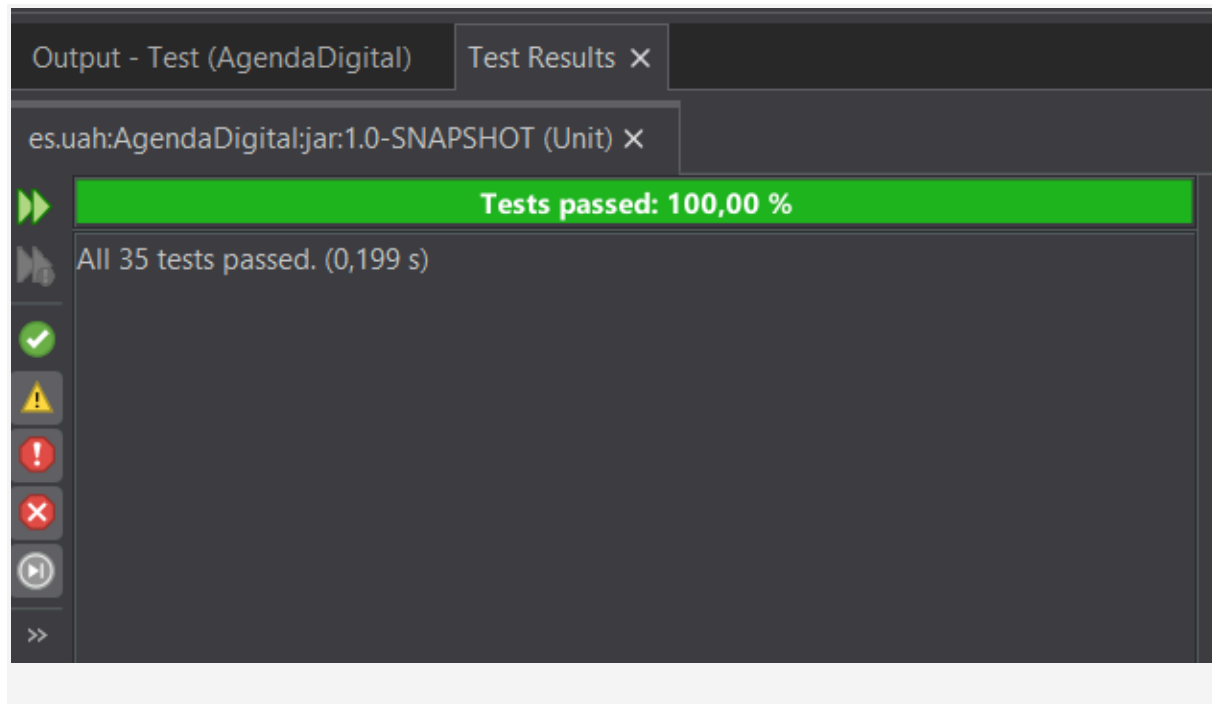
BUILD SUCCESS

---

Total time: 2.194 s

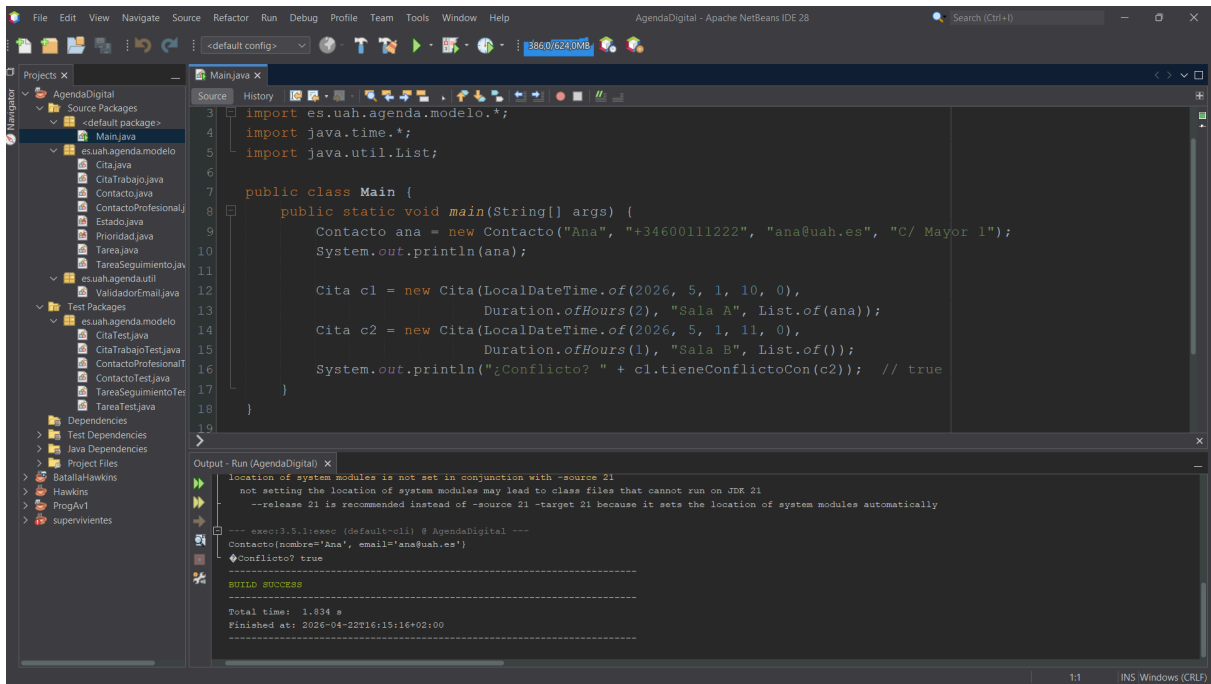
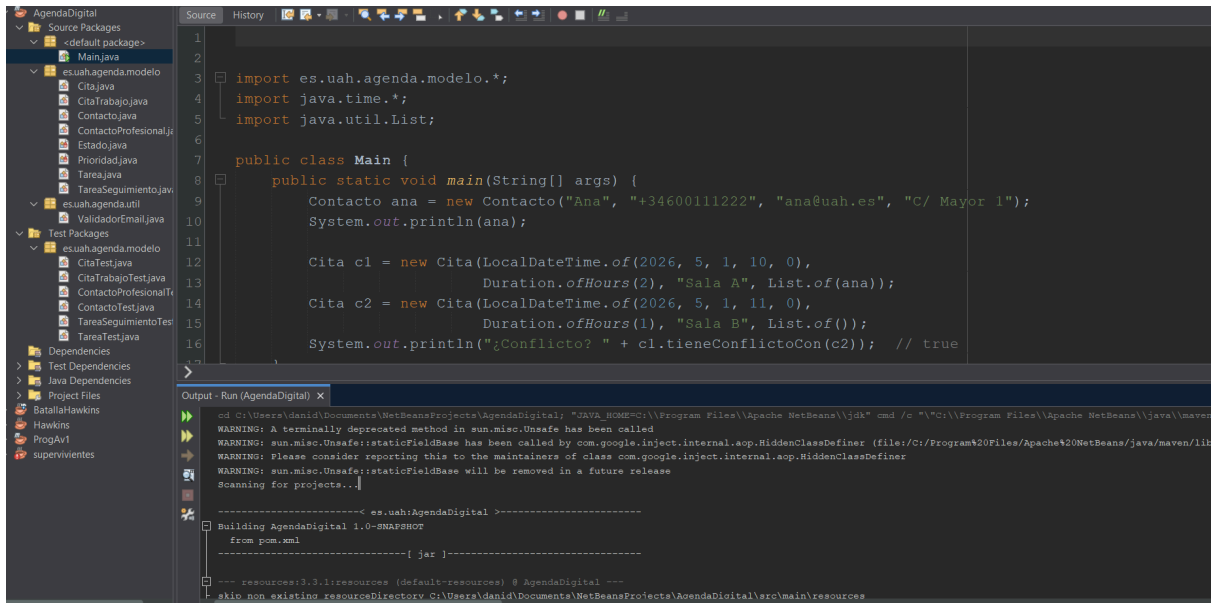
Finished at: 2026-04-22T15:58:25+02:00

---



Cobertura funcional de las pruebas:

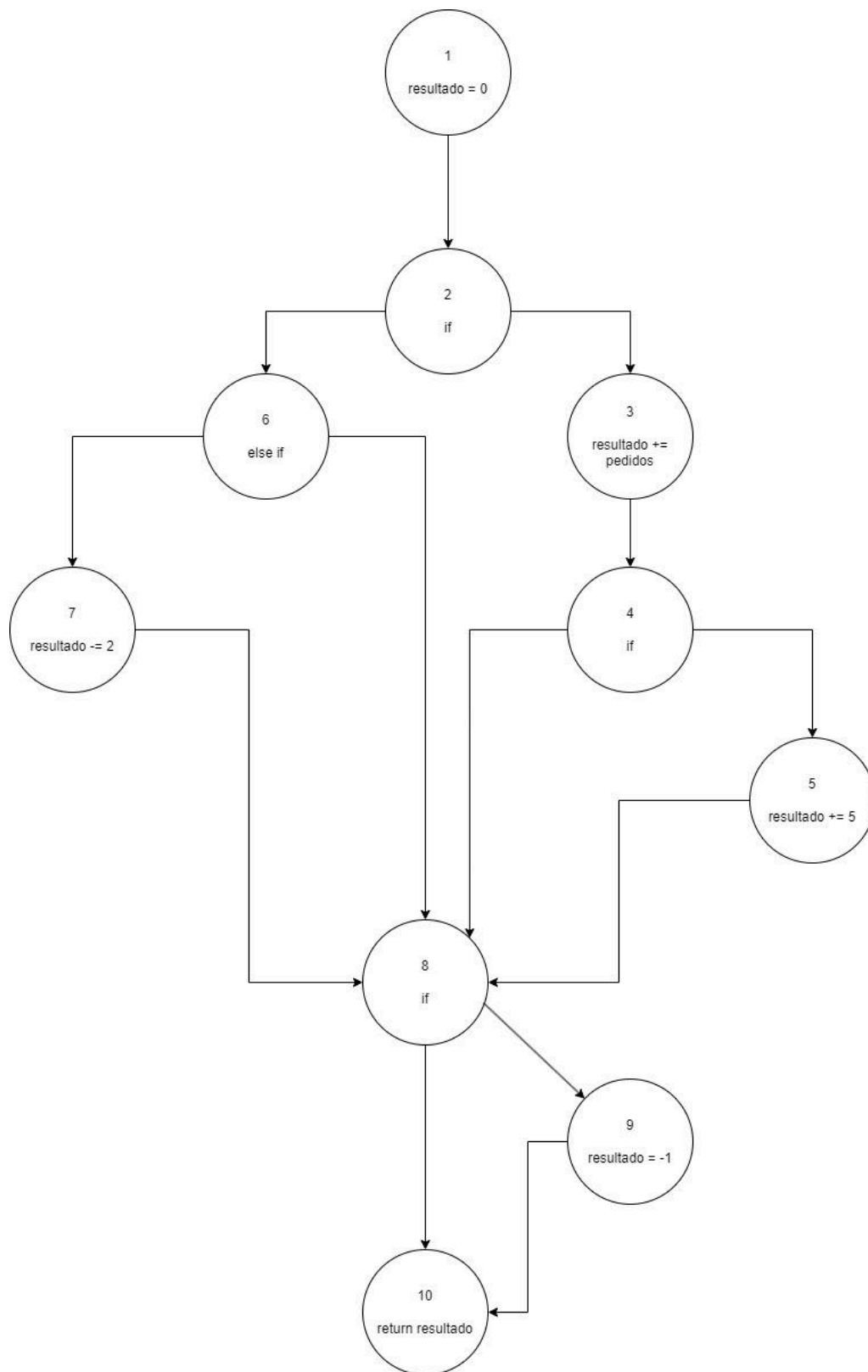
- Validación de entradas: nombre, email, teléfono, empresa, cargo, descripción, duración, fecha.
- Detección de conflictos temporales entre citas (solapamiento parcial, adyacencia, separación, auto-comparación, null).
- Inmutabilidad: `UnsupportedOperationException` al mutar las listas expuestas.
- Herencia: `ContactoProfesional` hereda correctamente validaciones de `Contacto`.
- Igualdad: `Contacto.equals/hashCode` basado exclusivamente en email.
- Semántica de negocio: `TareaSeguimiento.estaVencida` respeta estados COMPLETADA y CANCELADA.
- Ordenación natural: Tarea ordena por prioridad descendente (CRITICA → BAJA).



## Ejercicio 2 resuelto

```
public int gestionInventario(int stock, int pedidos, boolean urgente, boolean proveedorActivo) {  
    int resultado = 0;  
  
    if (stock < 10 && pedidos > 0) {  
        resultado += pedidos;  
        if (urgente) {  
            resultado += 5; // margen de urgencia  
        }  
    } else if (stock >= 10) {  
        resultado -= 2; // pequeña penalización por sobrealmacenaje  
    }  
  
    if (!proveedorActivo) {  
        resultado = -1; // error en la gestión  
    }  
  
    return resultado;  
}
```

## Cálculo de complejidad ciclomática



Al realizar el diagrama de nodos para el código dado, podemos ver que tenemos 10 nodos y 13 aristas.

Por lo tanto, usando la fórmula de McCabe:  $M = E - N + 2$ , donde E es el número de aristas y N es el número de nodos. Nos sale que  $M = 13 - 10 + 2 = 5$ .

La complejidad ciclomática del código es 5.

## **Caminos básicos del código**

Gracias al diagrama realizado y el conocimiento de que la complejidad es 5. Podemos concluir que tenemos 5 caminos básicos importantes por lo que podemos recorrer este código:

- 1 > 2 > (...) > 8 > 9 > 10 (La razón por la que hay (...) aquí es debido a que no importa que camino se toma por el medio, si se toma el nodo 9 el resultado siempre es -1)
- 1 > 2 > 3 > 4 > 5 > 8 > 10
- 1 > 2 > 3 > 4 > 8 > 10
- 1 > 2 > 6 > 7 > 8 > 10
- 1 > 2 > 6 > 8 > 10

## Ejercicio 3 resuelto

```
public int gestionInventario(int stock, int pedidos, boolean urgente, boolean proveedorActivo) {  
    int resultado = 0;  
    if (stock < 10 && pedidos > 0) {  
        resultado += pedidos;  
        if (urgente) {  
            resultado += 5; // margen de urgencia  
        }  
    } else if (stock >= 10) {  
        resultado -= 2; // pequeña penalización por sobrealmacenaje  
    }  
    if (!proveedorActivo) {  
        resultado = -1; // error en la gestión  
    }  
    return resultado;  
}
```

## Diseño de alto nivel mediante ingeniería inversa

El código aportado pertenece a una función llamada gestión Inventario. Revisando el contenido del código, parece que está relacionado con el stock de un almacén y los pedidos que hay pendientes.

## Funcionamiento

Recibe cuatro datos:

- stock: las unidades que hay en el almacén.
- pedidos: los pedidos que están pendientes.
- urgente: si hay prisa o no.
- proveedorActivo: si el proveedor está disponible o no.

Y devuelve un número entero. Según las condiciones, puede ser:

- La suma de los pedidos (y a veces +5 si es urgente).
- Un -2 si hay demasiado stock.
- Un -1 si el proveedor no está activo.

La función va comprobando siguiendo orden secuencial, diferenciando los caminos según la condición If:

- Primero mira si hay poco stock (menos de 10) y además hay pedidos. Si es así, suma los pedidos al resultado.
- Dentro de esa misma comprobación, si también es urgente, suma 5 más.

- Si no se cumple lo anterior, mira si el stock es alto (10 o más). Entonces resta 2.
- Al final, haga lo que haga, si el proveedor no está activo, el resultado pasa a ser -1 directamente.

## Lógica de Negocio

Viendo el funcionamiento, podemos abstraer que la secuencia de este método tiene como idea principal que cuando falta stock y hay pedidos, los pedidos se suman (quizás para saber cuánto hay que pedir). Si hay prisa, se añade un extra. Si hay demasiado stock, se penalizará restando 2. Pero si el proveedor no está activo, todo lo anterior se ignora y se devuelve un error (-1)

## Resultados y justificación de pruebas de caja blanca

Como ya hemos visto en el Ejercicio 2, sabemos que la complejidad ciclomática de la función es 5. Esto significa que hay 5 casos de prueba, uno por cada camino diferente que tenemos que probar para asegurarnos de que recorremos todo el código.

### Los 5 caminos y sus casos de prueba

| CAMINO                     | FUNCIONAMIENTO                                                       | VALORES DE PRUEBA    | RESULTADO |
|----------------------------|----------------------------------------------------------------------|----------------------|-----------|
| 1 > 2 > (...) > 8 > 9 > 10 | Si el proveedor no está activo, da igual lo demás, sale -1.          | (5, 3, false, false) | -1        |
| 1 > 2 > 3 > 4 > 5 > 8 > 10 | Si el stock es bajo, hay pedidos y es urgente, suma pedidos +5.      | (5, 3, true, true)   | 8         |
| 1 > 2 > 3 > 4 > 8 > 10     | Si el stock es bajo, hay pedidos pero no urgente, solo suma pedidos. | (5, 3, false, true)  | 3         |
| 1 > 2 > 6 > 7 > 8 > 10     | Si el stock es alto (10 o más), devuelve -2.                         | (10, 0, false, true) | -2        |
| 1 > 2 > 6 > 8 > 10         | Si el stock es alto aunque haya pedidos, devuelve -2.                | (10, 5, false, true) | -2        |

- C1 comprueba que la condición del proveedor tiene prioridad total y no depende de los resultados anteriores.
- C2 y C3 comprueban que la rama de cuando hay stock bajo funciona bien tanto con urgencia como sin ella.
- C4 y C5 comprueban que la rama de cuando hay stock alto funciona, tanto si hay pedidos como si no (aunque el código ignora los pedidos aquí).

## Resultados y justificación de pruebas de caja negra

Para comprobar el funcionamiento basta con realizar 10 pruebas para cubrir las particiones esperadas. Las fronteras importantes a tener en cuenta se encuentran en stock=10 y pedidos=0.

### Particiones de Equivalencia

| Parámetro       | Grupo válido                  | Ejemplos    |
|-----------------|-------------------------------|-------------|
| stock           | Bajo (stock < 10)             | -5, 0, 5, 9 |
| stock           | Alto (stock >= 10)            | 10, 15, 100 |
| pedidos         | Hay pedidos (pedidos > 0)     | 1, 5, 50    |
| pedidos         | No hay pedidos (pedidos <= 0) | 0, -1, -10  |
| urgente         | true                          | true        |
| urgente         | false                         | false       |
| proveedorActivo | true                          | true        |
| proveedorActivo | false                         | false       |

### Casos de prueba

| stock | pedidos | urgente | urgente | Salida esperada |
|-------|---------|---------|---------|-----------------|
| 5     | 3       | false   | true    | 3               |
| 5     | 3       | true    | true    | 8               |
| 5     | 0       | false   | true    | 0               |

|     |     |       |       |    |
|-----|-----|-------|-------|----|
| 5   | -1  | false | true  | 0  |
| 9   | 3   | false | true  | 3  |
| 10  | 3   | false | true  | -2 |
| 11  | 3   | false | true  | -2 |
| 5   | 3   | false | false | -1 |
| 10  | 3   | false | false | -1 |
| 100 | 100 | true  | true  | -2 |

Los casos comprueban el cumplimiento de cada una de las posibilidades por partición:

- Stock bajo y alto (casos 1, 5, 6, 7)
- Pedidos positivos, cero y negativos (casos 1, 3, 4)
- Urgente true y false (casos 1 y 2)
- Proveedor activo e inactivo (casos 1 y 8)
- Los valores límite cerca de stock=10 (casos 5, 6, 7)
- Combinaciones importantes como proveedor inactivo con distintas situaciones (casos 8 y 9)